

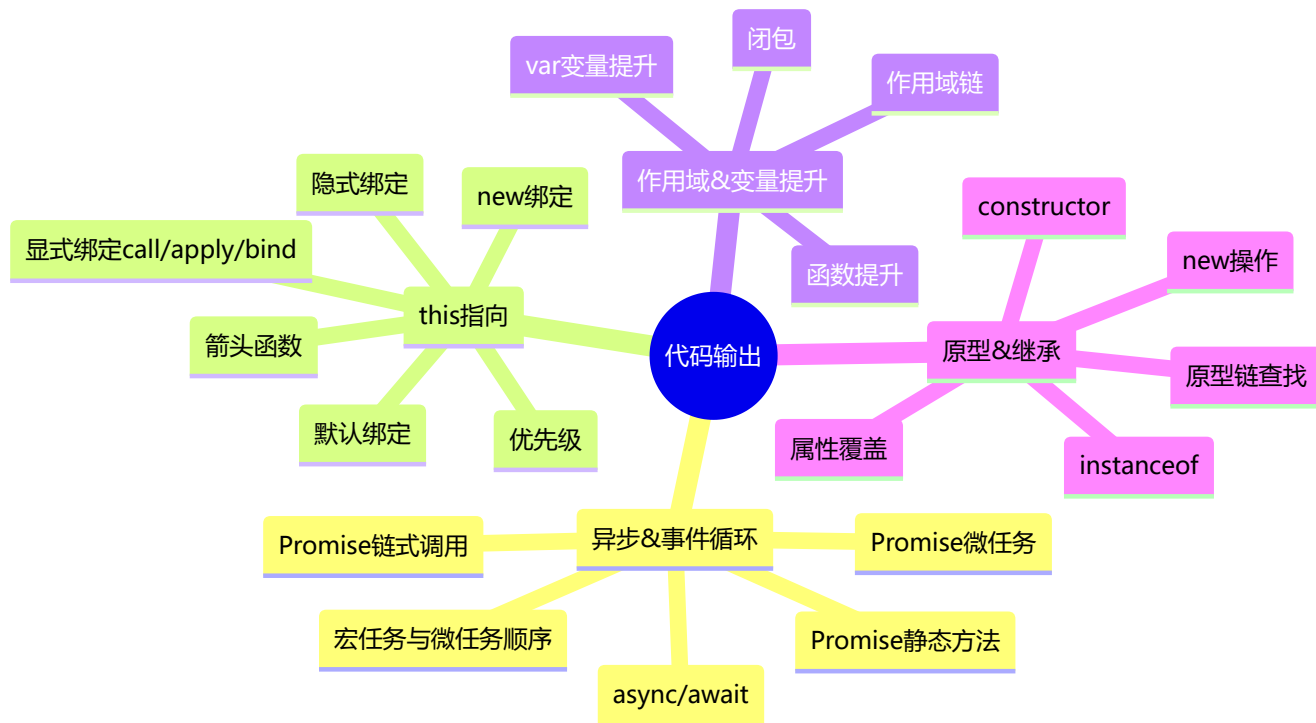


代码输出题详解版（含 Mermaid 图解）

🚀 前端面试必备 - 代码输出题全面梳理 | 建议收藏 ⭐



知识脑图

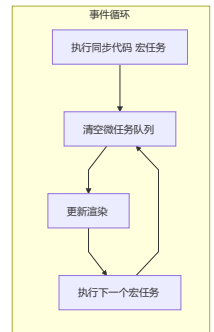


目录

- [🕒 一、异步 & 事件循环](#)
- [🎯 二、this 指向](#)
- [🔗 三、作用域 & 变量提升](#)
- [🧬 四、原型 & 继承](#)
- [❓ 五、空值合并 & 可选链](#)
- [⚡ 六、Promise 进阶方法](#)
- [🔄 七、Generator & 迭代器](#)
- [👤 八、Proxy & Reflect](#)
- [🏠 九、Class 语法 & 继承](#)
- [📁 十、Map / Set / WeakMap / WeakSet](#)



一、异步 & 事件循环



1 代码输出结果

💡 **要点：** Promise 构造函数是**同步执行**的，`.then()` 注册的回调是**微任务**。若 Promise 状态一直为 `pending`，则 `.then()` 永远不会执行。

```
const promise = new Promise((resolve, reject) => {
  console.log(1);
  console.log(2);
});
promise.then(() => {
  console.log(3);
});
console.log(4);
```

输出结果如下： 1 2 4

`promise.then` 是微任务，它会在所有的宏任务执行完之后才会执行，同时需要 `promise` 内部的状态发生变化，因为这里内部没有发生变化，一直处于 `pending` 状态，所以不输出 3。

2 代码输出结果

💡 **要点：** Promise 构造函数**同步执行**并立即改变状态，`.then()` 是微任务延迟执行。注意 `promise1` 此时已是 `resolved`，而 `promise2` 还是 `pending`（因为 `then` 还没执行）。

```
const promise1 = new Promise((resolve, reject) => {
  console.log('promise1')
  resolve('resolve1')
})
const promise2 = promise1.then(res => {
  console.log(res)
})
console.log('1', promise1);
console.log('2', promise2);
```

输出结果如下：

```
promise1
1 Promise{<resolved>: resolve1}
2 Promise{<pending>}
resolve1
```

3 代码输出结果

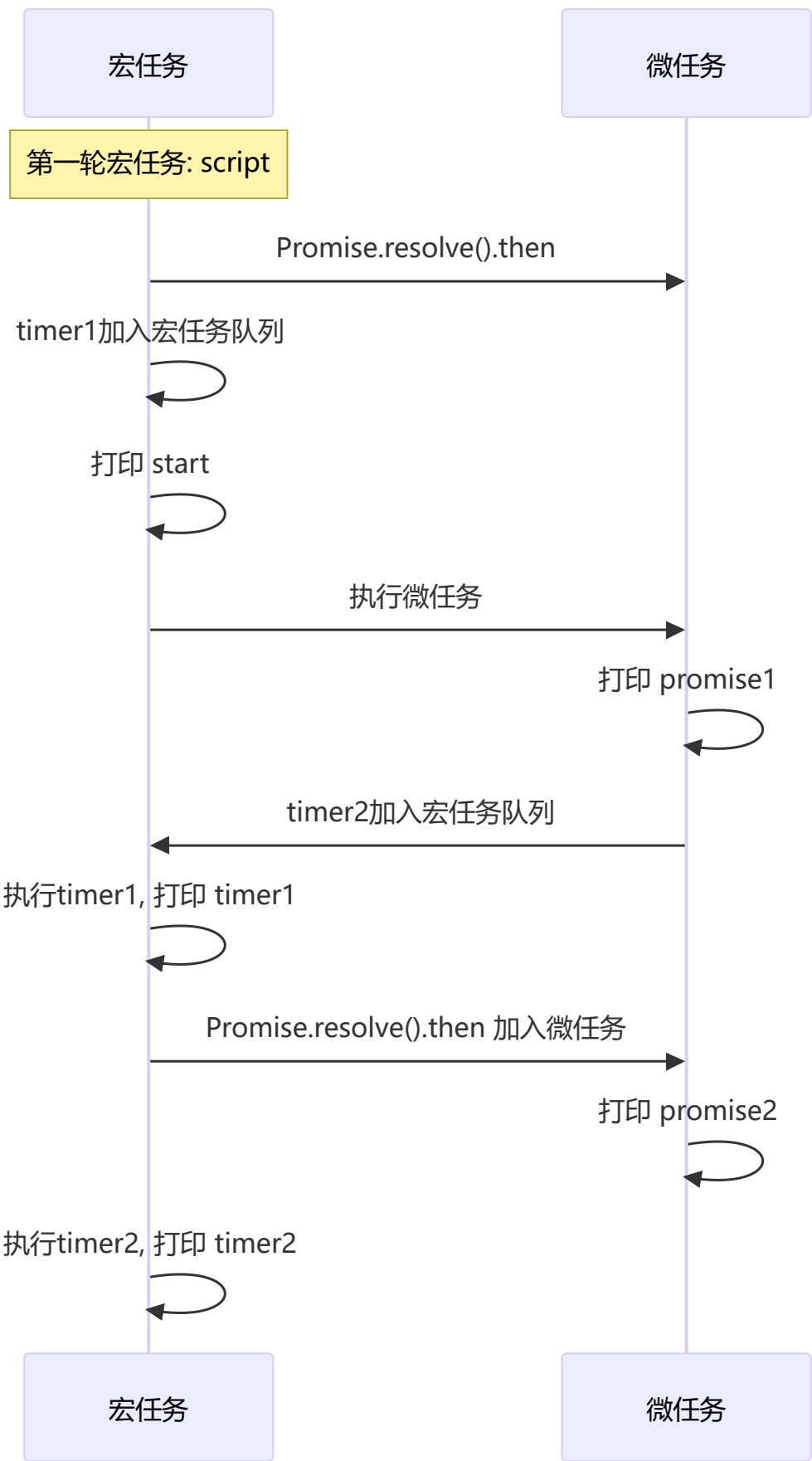
💡 **要点:** `setTimeout` 是**宏任务**，其中的 `resolve` 要在宏任务执行时才会调用，而 `.then()` 微任务需要等 `resolve` 之后才注册到微任务队列。

```
const promise = new Promise((resolve, reject) => {
  console.log(1);
  setTimeout(() => {
    console.log("timerStart");
    resolve("success");
    console.log("timerEnd");
  }, 0);
  console.log(2);
});
promise.then((res) => {
  console.log(res);
});
console.log(4);
```

输出结果如下: 1 2 4 timerStart timerEnd success

4 代码输出结果

💡 **要点:** 理解 **宏任务 (MacroTask)** 与 **微任务 (MicroTask)** 的交替执行规则: 每一轮宏任务执行完毕后, 会清空当前所有的微任务, 然后进行下一轮宏任务。



5 代码输出结果

💡 **要点：** Promise 的状态一旦变更就不可逆转（从 pending 变为 resolved/rejected 后即锁定），后续的 resolve/reject 调用均被忽略。

```
const promise = new Promise((resolve, reject) => {
  resolve('success1');
  reject('error');
  resolve('success2');
});
promise.then((res) => {
  console.log('then:', res);
}).catch((err) => {
  console.log('catch:', err);
})
```

输出: then: success1

Promise的状态在发生变化之后，就不会再发生变化。

6 代码输出结果

💡 **要点：** .then() 接受的参数必须是函数，传入非函数（如数字、对象等）会导致值透传——前一个 Promise 的值直接传递到下一个 .then()。

```
Promise.resolve(1)
  .then(2)
  .then(Promise.resolve(3))
  .then(console.log)
```

输出: 1

then方法接受的参数必须是函数，如果传递的并非是一个函数，实际上会将其解释为then(null)，这就会导致前一个 Promise的结果会透传到后面。

7 代码输出结果

💡 **要点：** .then() 会返回一个新的 Promise。初始状态为 pending，等到 then 中的回调执行完毕后，新 Promise 才会改变状态。throw 会使新 Promise 变为 rejected。

```
const promise1 = new Promise((resolve, reject) => {
  setTimeout(() => { resolve('success') }, 1000)
})
const promise2 = promise1.then(() => {
  throw new Error('error!!!')
})
console.log('promise1', promise1)
console.log('promise2', promise2)
setTimeout(() => {
  console.log('promise1', promise1)
  console.log('promise2', promise2)
}, 2000)
```

输出:

```
promise1 Promise {<pending>}
promise2 Promise {<pending>}
Uncaught (in promise) Error: error!!!
promise1 Promise {<fulfilled>: "success"}
promise2 Promise {<rejected>: Error: error!!}
```

8 代码输出结果

💡 要点: Promise 链中, `.catch()` 只在前面的 Promise 为 `rejected` 时才会执行。未出错时 `.catch()` 会被跳过, 值直接传给下一个 `.then()`。

```
Promise.resolve(1)
  .then(res => { console.log(res); return 2; })
  .catch(err => { return 3; })
  .then(res => { console.log(res); });
```

输出: 1 2

9 代码输出结果

💡 要点: 在 `.then()` 中 `return new Error(...)` 不会触发 `.catch()`, 因为返回的 Error 对象会被当作普通值包裹成 resolved Promise。要让 catch 捕获, 需要使用 `throw new Error(...)` 或 `return Promise.reject(...)`。

```
Promise.resolve().then(() => {
  return new Error('error!!!')
}).then(res => {
  console.log("then: ", res)
}).catch(err => {
  console.log("catch: ", err)
})
```

输出: "then: " "Error: error!!!"

返回任意一个 promise 的值都会被包裹成 promise 对象。

10 代码输出结果

💡 要点: `.then()` 或 `.catch()` 的返回值**不能是 Promise 自身**, 否则会形成**死循环** (Chaining cycle), 抛出 `TypeError`。

```
const promise = Promise.resolve().then(() => {  
  return promise;  
})  
promise.catch(console.err)
```

输出: `Uncaught (in promise) TypeError: Chaining cycle detected for promise`

`.then` 或 `.catch` 返回的值不能是 promise 本身, 否则会造成死循环。

1 1 代码输出结果

⚠ 易错点: 与第 6 题类似, `.then()` 传非函数导致**值透传**。 `Promise.resolve(3)` 虽然传给了 `.then()`, 但 `.then()` 期望的是函数而非 Promise 对象, 所以不会执行。

```
Promise.resolve(1)  
  .then(2)  
  .then(Promise.resolve(3))  
  .then(console.log)
```

输出: `1`

传入非函数则会发生**值透传**。

1 2 代码输出结果

💡 要点: `.then()` 的**第二个参数** (错误处理回调) 和 `.catch()` 都能捕获错误, 但被第二个参数捕获的错误**不会再传递给后续的 .catch()**。

```
Promise.reject('err!!!')  
  .then((res) => { console.log('success', res) },  
        (err) => { console.log('error', err) })  
  .catch(err => { console.log('catch', err) })
```

输出: `error err!!!`

错误直接被then的第二个参数捕获, 就不会被catch捕获了。

1 3 代码输出结果

💡 要点: `.finally()` 不管 Promise 成功还是失败都会执行, 但它不接受任何参数, 且返回值不会影响链中下一个 `.then()` 收到的值 (仍为前一个 Promise 的值)。

```
Promise.resolve('1')
  .then(res => { console.log(res) })
  .finally(() => { console.log('finally') })
Promise.resolve('2')
  .finally(() => {
    console.log('finally2')
    return '我是finally2返回的值'
  })
  .then(res => { console.log('finally2后面的then函数', res) })
```

输出: 1 finally2 finally finally2后面的then函数 2

`.finally()` 不管 Promise 对象最后的状态如何都会执行, 不接受任何参数, 返回默认是上一次的 Promise 对象值。

1 4 代码输出结果

💡 要点: `Promise.all` 接收一个 Promise 数组, 所有 Promise 都成功后才进入 `.then()`, 结果数组按传入顺序返回。多个 Promise 同步启动, 但各自异步执行。

```
function runAsync (x) {
  const p = new Promise(r => setTimeout(() => r(x, console.log(x)), 1000))
  return p
}
Promise.all([runAsync(1), runAsync(2), runAsync(3)]).then(res => console.log(res))
```

输出: 1 2 3 [1, 2, 3]

三个函数是同步执行的, 结果和函数的执行顺序一致。

1 5 代码输出结果

💡 要点: `Promise.all` 遵循快速失败原则——只要有一个 Promise 被 `rejected`, 整体立即进入 `.catch()`, 但其他 Promise 仍会继续执行 (只是其结果被忽略)。


```
function runAsync (x) {
  const p = new Promise(r => setTimeout(() => r(x, console.log(x)), 1000))
  return p
}
function runReject (x) {
  const p = new Promise((res, rej) => setTimeout(() => rej(`Error: ${x}`, console.log(x)),
1000 * x))
  return p
}
Promise.all([runAsync(1), runReject(4), runAsync(3), runReject(2)])
  .then(res => console.log(res))
  .catch(err => console.log(err))
```

输出:

```
// 1s后输出
1 3
// 2s后输出
2 Error: 2
// 4s后输出
4
```

catch捕获到了第一个错误 runReject(2) 的结果。

1 6 代码输出结果

💡 **要点:** `Promise.race` 返回**第一个状态变更** (无论是 resolved 还是 rejected) 的 Promise 结果。其他慢的 Promise 虽然会继续执行, 但结果不会被消费。

```
function runAsync (x) {
  const p = new Promise(r => setTimeout(() => r(x, console.log(x)), 1000))
  return p
}
Promise.race([runAsync(1), runAsync(2), runAsync(3)])
  .then(res => console.log('result: ', res))
  .catch(err => console.log(err))
```

输出: 1 'result: ' 1 2 3

then只会捕获第一个成功的方法。

1 7 async/await 输出

💡 **要点:** `await` 后面的代码相当于包裹在 `new Promise` 的构造函数中同步执行, 而 `await` 下面的代码相当于放在 `.then()` 中的**微任务**, 会等待当前宏任务中的同步代码执行完毕后执行。

```

async function async1() {
  console.log("async1 start");
  await async2();
  console.log("async1 end");
}
async function async2() {
  console.log("async2");
}
async1();
console.log('start')

```

输出: `async1 start async2 start async1 end`

await 后面的语句相当于放到了new Promise中，下一行及之后的语句相当于放在Promise.then中。

1 8 async/await + Promise + setTimeout 综合

💡 **要点：**经典面试综合题，考察三者的执行顺序：**同步代码** → **微任务**（await 后续 + Promise.then） → **宏任务**（setTimeout）。注意 `await` 会先执行右侧函数，然后将后续代码放入微任务队列。

```

async function async1() {
  console.log("async1 start");
  await async2();
  console.log("async1 end");
}
async function async2() {
  console.log("async2");
}
console.log("script start");
setTimeout(function() { console.log("setTimeout"); }, 0);
async1();
new Promise(resolve => {
  console.log("promise1");
  resolve();
}).then(function() { console.log("promise2"); });
console.log('script end')

```

输出: `script start async1 start async2 promise1 script end async1 end promise2 setTimeout`

1 9 Promise 嵌套

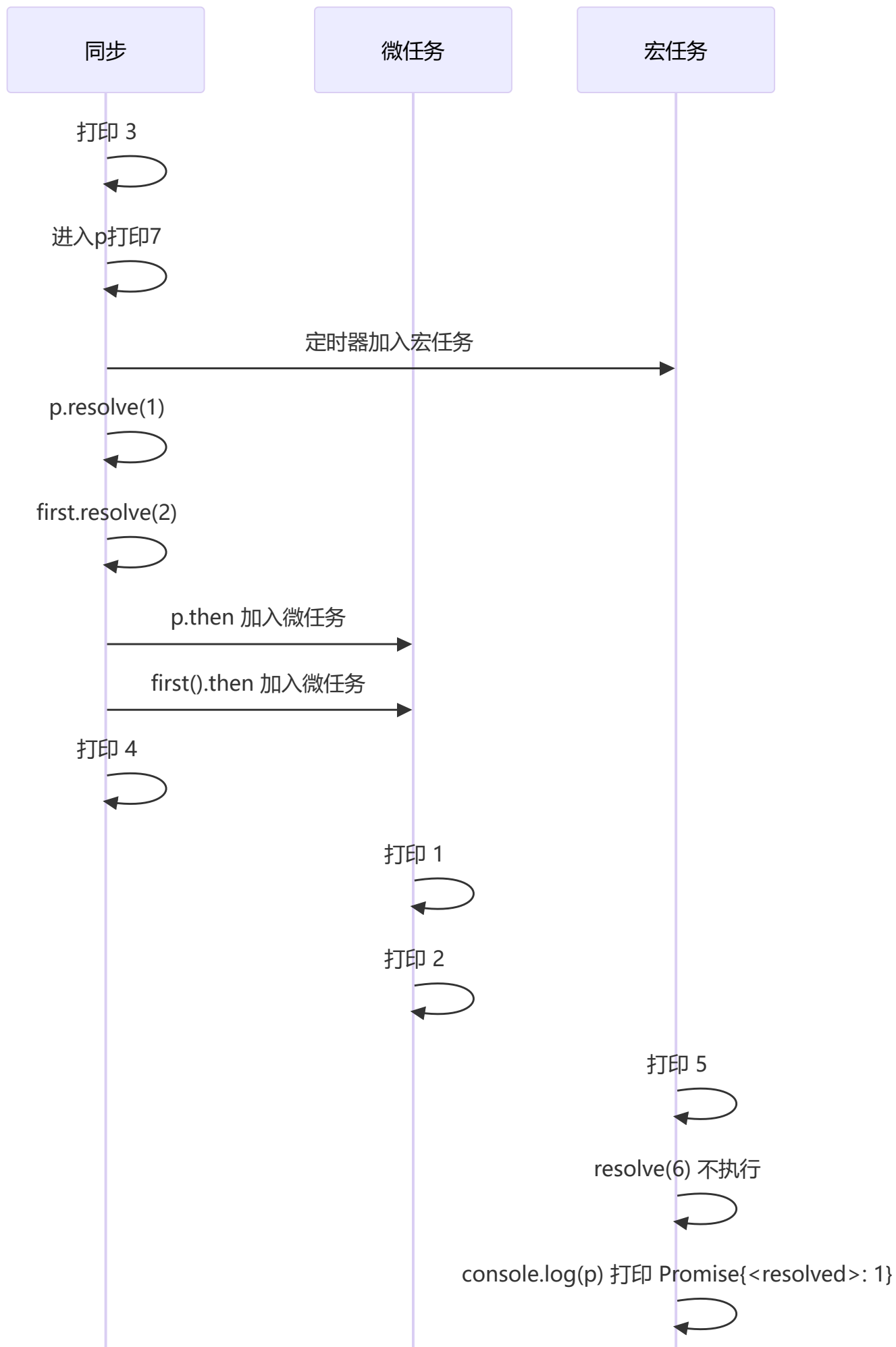
💡 **要点：**Promise 的 `resolve` 是同步调用的，但 `.then()` 是微任务。已经 resolve 的 Promise 后续再 resolve/reject 无效，所以 `setTimeout` 中的 `resolve(6)` 不会产生效果。

```

const first = () => (new Promise((resolve, reject) => {
  console.log(3);
  let p = new Promise((resolve, reject) => {
    console.log(7);
    setTimeout(() => {

```

```
        console.log(5);
        resolve(6);
        console.log(p)
    }, 0)
    resolve(1);
});
resolve(2);
p.then((arg) => { console.log(arg); });
});
first().then((arg) => { console.log(arg); });
console.log(4);
```



同步

微任务

宏任务

二、this 指向

1 代码输出结果

💡 要点：箭头函数不绑定 `this`，它的 `this` 继承自定义时的外层作用域（此处为全局/模块作用域）。
`call/apply/bind` 无法改变箭头函数的 `this` 指向。

```
var a = 10
var obj = {
  a: 20,
  say: () => {
    console.log(this.a)
  }
}
obj.say()
var anotherObj = { a: 30 }
obj.say.apply(anotherObj)
```

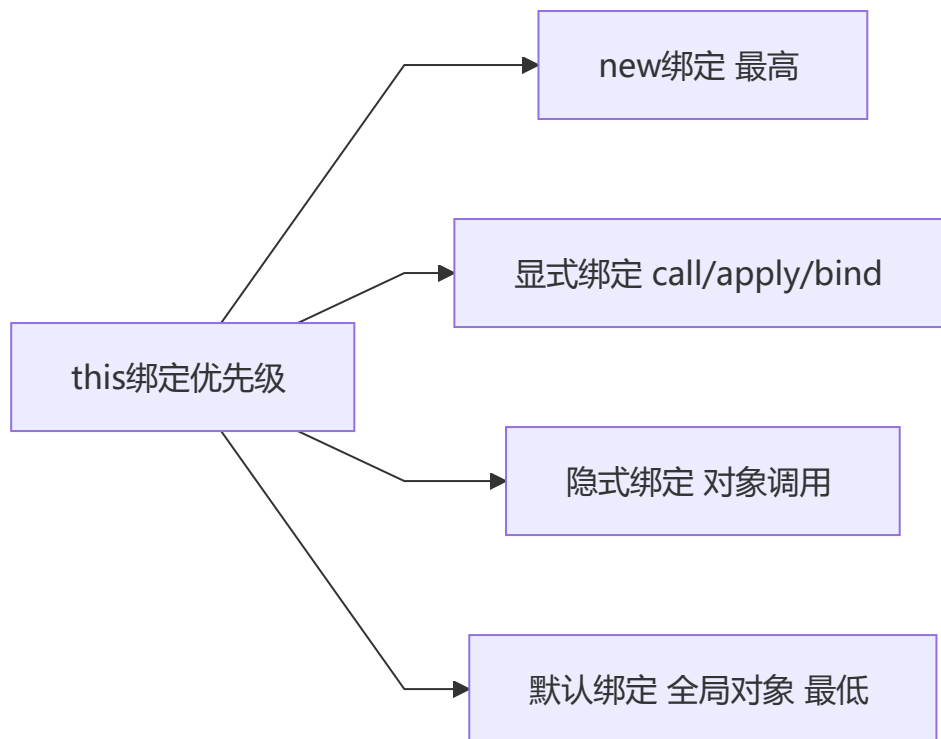
输出：10 10

箭头函数不绑定`this`，它的`this`来自其父级所处的上下文。

如果是普通函数，`say`方法中的`this`就会指向他所在的对象，输出：20 30。

2 this 绑定优先级

💡 要点：`this` 的 4 种绑定规则的优先级：**new 绑定** > **显式绑定** (`call/apply/bind`) > **隐式绑定** (对象调用) > **默认绑定** (全局对象)。bind 返回的函数被 new 时，bind 绑定失效。



```
function foo(something){
  this.a = something
}
var obj1 = {}
var bar = foo.bind(obj1);
bar(2);
console.log(obj1.a); // 2

var baz = new bar(3);
console.log(obj1.a); // 2
console.log(baz.a); // 3
```

结论：new绑定 > 显式绑定 > 隐式绑定 > 默认绑定

3 综合 this 题目

💡 要点：fn() 是默认绑定，this 指向全局（或 undefined 严格模式）。arguments[0]() 是隐式绑定，this 指向 arguments 对象，因此 this.length 等于传入的参数个数。

```
var length = 10;
function fn() { console.log(this.length); }
var obj = {
  length: 5,
  method: function(fn) {
    fn();
    arguments[0]();
  }
};
obj.method(fn, 1);
```

输出: 10 2

- 第一次fn(), this指向window, 输出10
- 第二次arguments0, this指向arguments, 传了两个参数, 输出arguments长度为2

🔗 三、作用域 & 变量提升

1 代码输出结果

💡 要点: `var x = y = 1` 从右向左执行: `y = 1` 没有 `var` 声明, 成为全局变量; `var x` 是函数局部变量, 外部无法访问。

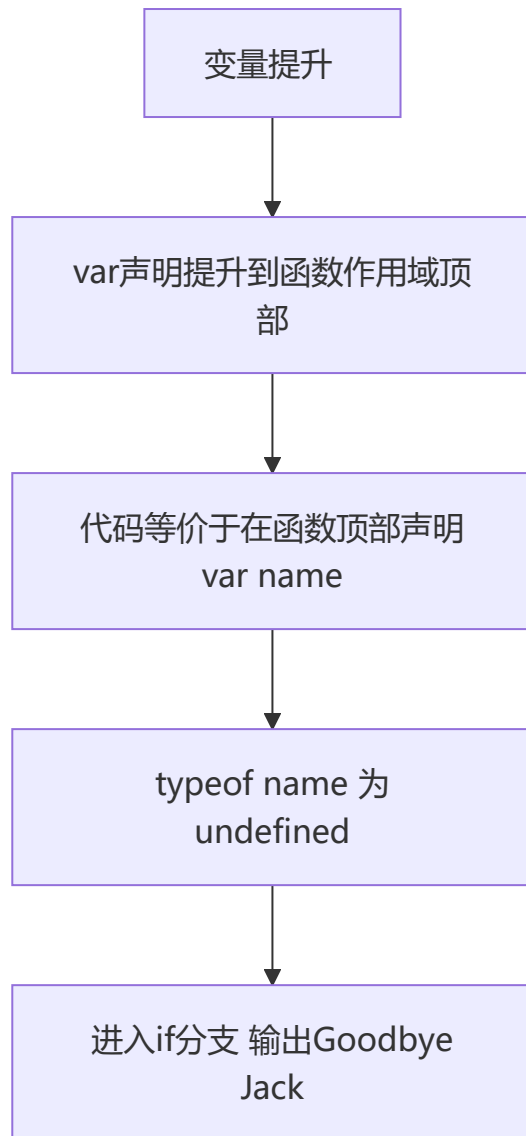
```
(function(){
  var x = y = 1;
})();
var z;
console.log(y); // 1
console.log(z); // undefined
console.log(x); // Uncaught ReferenceError: x is not defined
```

`var x = y = 1` 实际上是从右往左执行, `y = 1` 没有var声明, 所以是全局变量, x是局部变量。

2 变量提升

💡 要点: `var` 声明会被提升到函数作用域顶部。IIFE 内部的 `var friendName` 被提升到函数顶部, 导致 `typeof friendName` 为 `'undefined'` (覆盖了外部变量), 所以进入 if 分支。

```
var friendName = 'world';
(function() {
  if (typeof friendName === 'undefined') {
    var friendName = 'Jack';
    console.log('Goodbye ' + friendName);
  } else {
    console.log('Hello ' + friendName);
  }
})();
```



输出: Goodbye Jack

3 闭包经典题

💡 **要点:** 闭包中的内部函数**记住了外部函数的 `n` 值**。关键在于区分每次返回的对象中 `fun` 方法调用时传入的 `n` 是由哪个作用域提供的——这是闭包的核心考点。

```
function fun(n, o) {
  console.log(o)
  return {
    fun: function(m){
      return fun(m, n);
    }
  };
}

var a = fun(0); a.fun(1); a.fun(2); a.fun(3);
var b = fun(0).fun(1).fun(2).fun(3);
var c = fun(0).fun(1); c.fun(2); c.fun(3);
```


输出：

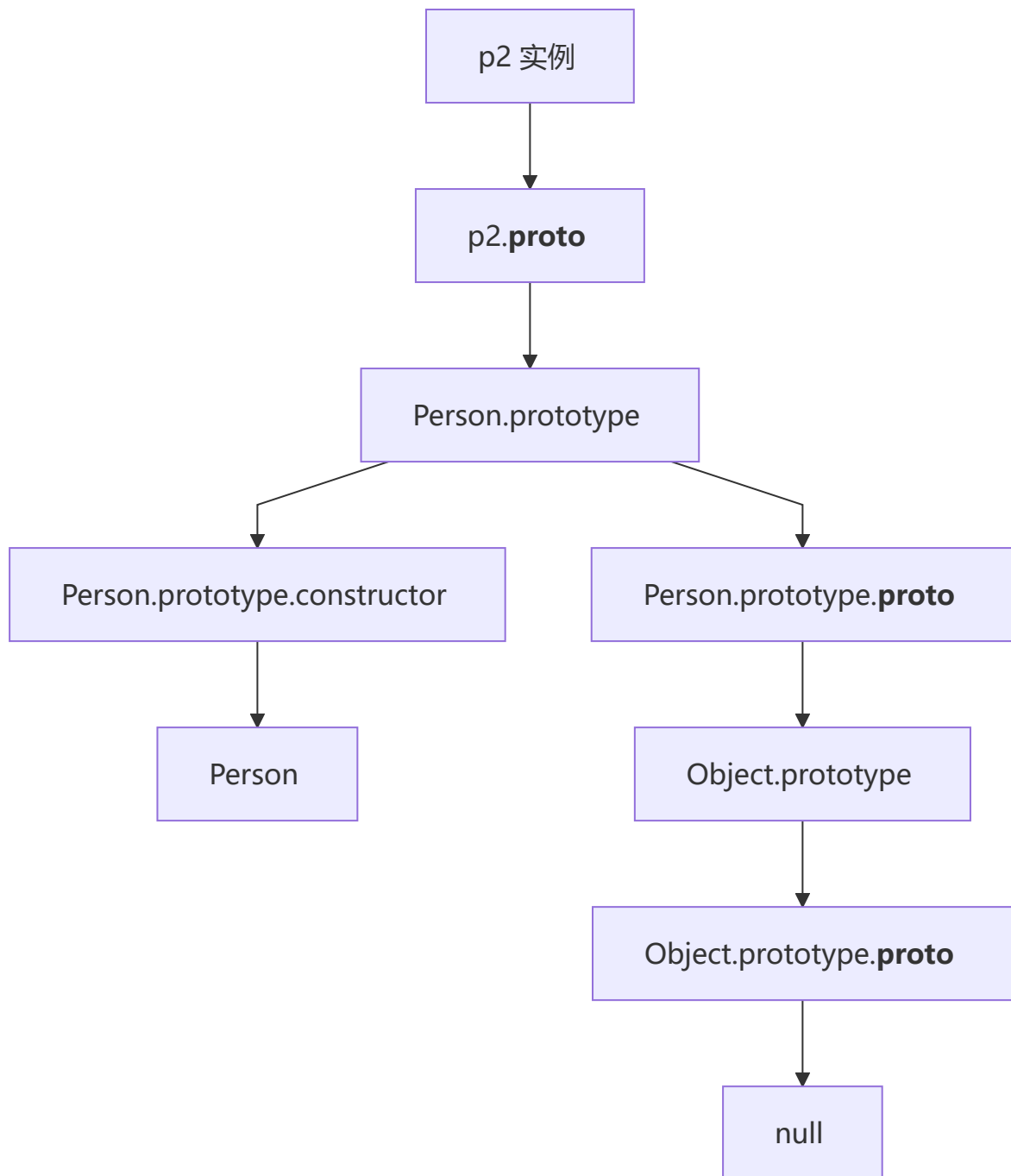
```
undefined 0 0 0
undefined 0 1 2
undefined 0 1 1
```

闭包中内部函数记住了外部函数的n值，每次调用都会记住当时传入的参数。

四、原型 & 继承

1 原型链

💡 要点：JavaScript 通过 `__proto__` 链接形成**原型链**，最顶层是 `Object.prototype.__proto__ === null`。`Person.prototype.constructor` 指向 `Person` 自身。



```
function Person(name) {  
    this.name = name  
}  
var p2 = new Person('king');  
console.log(p2.__proto__) //Person.prototype  
console.log(p2.__proto__.__proto__) //Object.prototype  
console.log(p2.__proto__.__proto__.__proto__) // null  
console.log(Person.constructor)//Function  
console.log(Function.prototype.__proto__)//Object.prototype
```

2 综合原型题

💡 要点：属性查找优先级：**实例自身属性 > 原型属性 > 静态方法**。 `new Foo()` 时函数内部的 `Foo.a` 覆盖了外部的静态方法，而 `this.a` 则成为实例属性。

```
function Foo(){
  Foo.a = function(){ console.log(1); }
  this.a = function(){ console.log(2); }
}
Foo.prototype.a = function(){ console.log(3); }
Foo.a = function(){ console.log(4); }

Foo.a();      // 4 - 静态方法
let obj = new Foo();
obj.a();      // 2 - 实例方法优先于原型
Foo.a();      // 1 - Foo函数内部覆盖了静态方法
```

3 原型继承

💡 要点：原型链继承的核心是 `SubType.prototype = new SuperType()`，子类原型指向父类实例，从而能够访问父类原型上的方法。这是最经典的继承方式之一。

```
function SuperType(){
  this.property = true;
}
SuperType.prototype.getSupValue = function(){
  return this.property;
};
function SubType(){
  this.subproperty = false;
}
SubType.prototype = new SuperType();
SubType.prototype.getSubValue = function (){
  return this.subproperty;
};
var instance = new SubType();
console.log(instance.getSupValue()); // true
```

4 原型属性查找

💡 要点：**实例属性 > 原型属性**。 `new B()` 时 `this.n = 9999` 在实例上创建了自身属性，不会去原型查找。而 `new C()` 没有给 `this.n` 赋值，读取时沿原型链找到 `A.n`（已被 `A.n++` 改为 4400）。

```

var A = {n: 4399};
var B = function(){this.n = 9999};
var C = function(){var n = 8888};
B.prototype = A;
C.prototype = A;
var b = new B();
var c = new C();
A.n++;
console.log(b.n); // 9999 - 自身有n属性
console.log(c.n); // 4400 - 自身无n, 去原型找A.n为4400

```

? 五、空值合并 & 可选链

1 ?? vs || 对比

💡 要点: `||` 判断所有 falsy 值 (0、''、false、null、undefined、NaN) , 而 `??` 只判断 **null** 和 **undefined**。因此 `0 ?? 'default'` 返回 0, `0 || 'default'` 返回 'default'。

```

const a = 0;
const b = '';
const c = false;
console.log(a || 'default'); // ?
console.log(a ?? 'default'); // ?
console.log(b || 'default'); // ?
console.log(b ?? 'default'); // ?
console.log(c || 'default'); // ?
console.log(c ?? 'default'); // ?

```

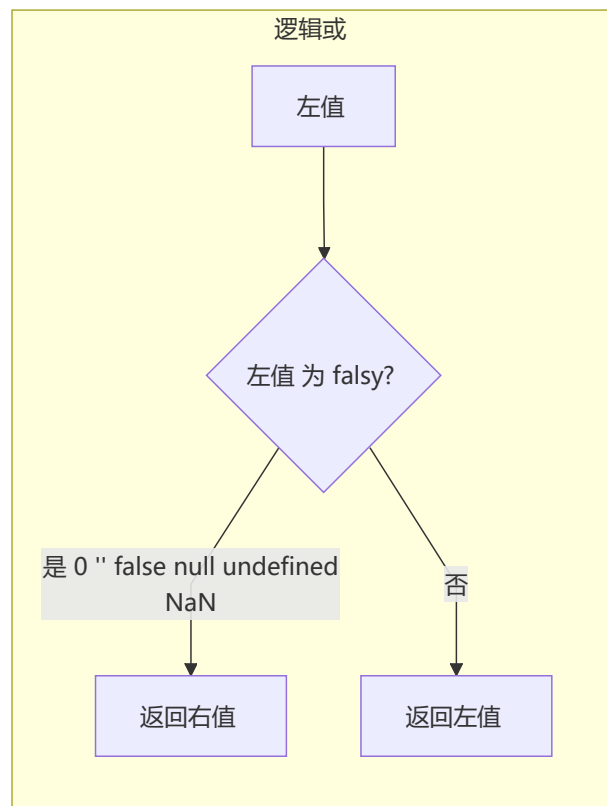
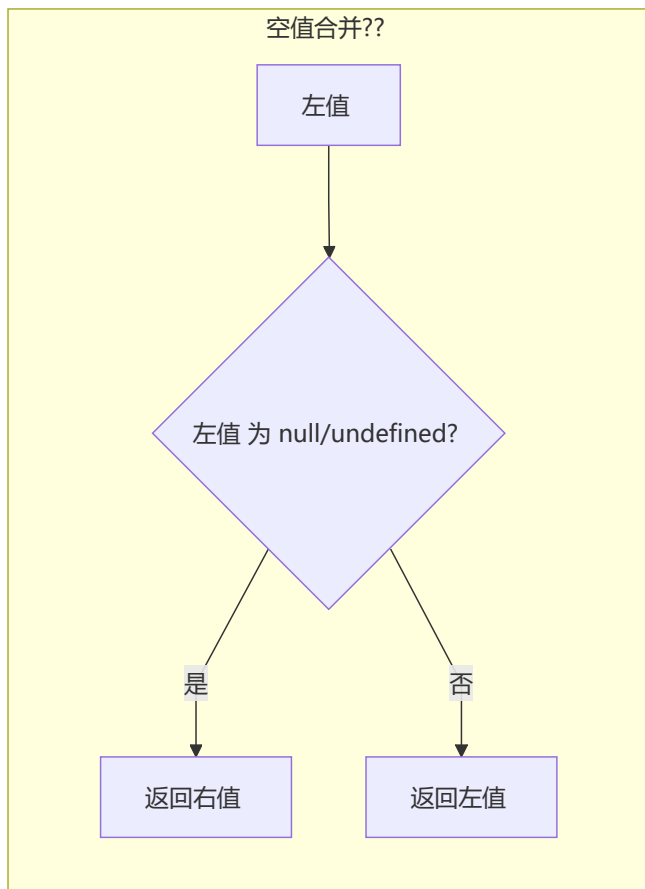
输出:

```

default
0
default

default
false

```



解析： `||` 判断所有 **falsy** 值 (0、"、false、null、undefined、NaN)，而 `??` 只判断 **null** 和 **undefined**。所以 `0 ?? 'default'` 返回 0，而 `0 || 'default'` 返回 'default'。同理，空字符串和 false 在 `??` 下被认为是有效值。

2 可选链 ?. 配合 ??

💡 要点： 可选链 `?.` 在遇到 `null/undefined` 时会**短路**返回 `undefined`，不会报错。结合 `??` 使用可以优雅地处理深层嵌套数据的默认值。

```
const obj = { a: { b: 0 } };
console.log(obj?.a?.b ?? 'default'); // ?
console.log(obj?.x?.y ?? 'default'); // ?
console.log(obj?.a?.c?.d ?? 'fallback'); // ?
```

输出：

```
0
default
fallback
```

解析： `obj?.a?.b` 正常访问到 0，`??` 判断 0 不是 null/undefined，所以返回 0。`obj?.x?.y` 中 x 不存在，可选链短路返回 undefined，`??` 检测到 undefined 返回 'default'。`obj?.a?.c?.d` 中 c 不存在，短路返回 undefined，返回 'fallback'。

3 混合运算

💡 **要点:** `??` 的优先级高于 `||`, 因此 `null ?? 'a' || 'b'` 等价于 `(null ?? 'a') || 'b'`。了解运算符优先级可以避免意外的逻辑错误。

```
const x = null ?? 'a' || 'b';
const y = (null ?? 'a') || 'b';
console.log(x); // ?
console.log(y); // ?
```

输出:

```
a
a
```

解析: `null ?? 'a' || 'b'` 等价于 `(null ?? 'a') || 'b'`, 因为 `??` 优先级高于 `||`。`null ?? 'a'` 返回 `'a'`, `'a' || 'b'` 返回 `'a'`, 所以两者结果相同。如果写成 `null ?? ('a' || 'b')`, 结果相同但运算顺序不同。

⚡ 六、Promise 进阶方法

1 Promise.allSettled

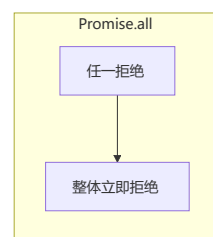
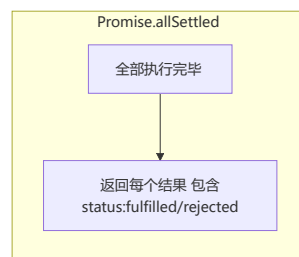
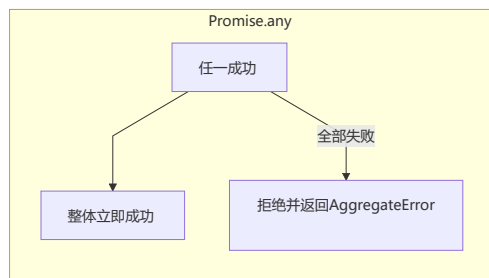
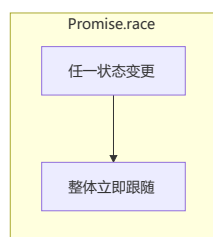
💡 **要点:** `Promise.allSettled` 不会短路, 会等待所有 Promise 完成 (无论成功或失败), 返回每个 Promise 的结果对象 `{status, value/reason}`。

```
const p1 = Promise.resolve(1);
const p2 = Promise.reject('err');
const p3 = new Promise(r => setTimeout(() => r(3), 100));

Promise.allSettled([p1, p2, p3]).then(console.log);
```

输出:

```
[
  { status: 'fulfilled', value: 1 },
  { status: 'rejected', reason: 'err' },
  { status: 'fulfilled', value: 3 }
]
```



解析： `Promise.allSettled` 不会短路，等待所有 promise 完成，返回每个 promise 的结果对象（包含 status 和 value/reason）。即使 p2 被拒绝，仍会等待 p3 的 100ms 定时器完成再输出。

2 Promise.any

💡 **要点：** `Promise.any` 返回**第一个成功**的 Promise 结果，忽略所有拒绝。只有**全部失败**时才会进入 catch，抛出 `AggregateError`。

```
const p1 = Promise.reject('err1');
const p2 = Promise.reject('err2');
const p3 = Promise.resolve('success');

Promise.any([p1, p2, p3]).then(console.log).catch(console.log);
```

输出：

```
success
```

解析： `Promise.any` 返回第一个成功的 promise 结果。p3 立即成功，所以输出 'success'。p1、p2 的拒绝被忽略。如果所有 promise 都失败，才会进入 catch。

3 Promise.any 全部失败

💡 **要点：** 当 `Promise.any` 的所有 Promise 都失败时，抛出 `AggregateError`，其 `errors` 属性是一个包含所有拒绝原因的数组。

```
const p1 = Promise.reject('err1');
const p2 = Promise.reject('err2');

Promise.any([p1, p2]).catch(err => {
  console.log(err.name);
  console.log(err.errors);
});
```

输出：

```
AggregateError
['err1', 'err2']
```

解析： 当 `Promise.any` 的所有 promise 都失败时，会抛出一个 `AggregateError` 类型的错误。`err.name` 为 'AggregateError'，`err.errors` 是一个数组，包含所有 promise 的拒绝原因。

七、Generator & 迭代器

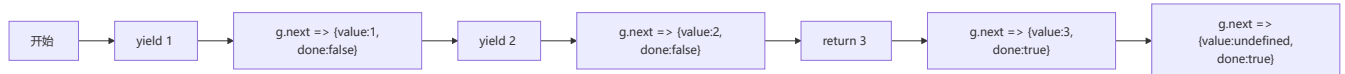
1 基础 Generator

💡 **要点：** Generator 函数通过 `yield` 暂停执行并返回值。`return` 也会返回值并将 `done` 置为 `true`。之后调用 `next()` 返回 `{value: undefined, done: true}`。

```
function* gen() {  
  yield 1;  
  yield 2;  
  return 3;  
}  
  
const g = gen();  
console.log(g.next());  
console.log(g.next());  
console.log(g.next());  
console.log(g.next());
```

输出：

```
{ value: 1, done: false }  
{ value: 2, done: false }  
{ value: 3, done: true }  
{ value: undefined, done: true }
```



解析： Generator 函数通过 `yield` 暂停执行并返回值。`return` 也会返回值，同时将 `done` 置为 `true`。之后再次调用 `next()` 返回 `{value: undefined, done: true}`。

2 Generator + yield 委托

💡 **要点：** `yield*` 表达式将执行委托给另一个 Generator，相当于内联展开被委托的 Generator 中的所有 `yield`。`[...gen1()]` 展开迭代器收集所有 `yield` 的值（不包含 `return`）。

```
function* gen1() {  
  yield 1;  
  yield* gen2();  
  yield 4;  
}  
  
function* gen2() {  
  yield 2;  
  yield 3;  
}  
  
console.log([...gen1()]);
```

输出：


```
[1, 2, 3, 4]
```

解析：`yield*` 表达式用于委托给另一个 generator。当执行到 `yield* gen2()` 时，进入 `gen2` 并依次 `yield` 2 和 3，然后回到 `gen1` 继续 `yield` 4。`[...gen1()]` 展开迭代器，收集所有 `yield` 的值（不包括 `return` 的值）。

3 async Generator

💡 要点：异步 Generator 可同时使用 `await` 和 `yield`：先用 `await` 等待 Promise，再 `yield` 结果。`for await...of` 用于消费异步迭代器。

```
async function* asyncGen() {
  yield await Promise.resolve(1);
  yield await Promise.resolve(2);
}
(async () => {
  for await (const val of asyncGen()) {
    console.log(val);
  }
})();
```

输出：

```
1
2
```

解析：异步 Generator 可以同时使用 `await` 和 `yield`。每次迭代时，先用 `await` 等待 promise 完成，再 `yield` 结果。`for await...of` 用于消费异步迭代器。

八、Proxy & Reflect

1 Proxy 基础

💡 要点：Proxy 通过陷阱 (trap) 拦截对象的基础操作。`get` 陷阱拦截属性读取，`set` 陷阱拦截属性赋值，需返回 `true` 表示成功。修改 proxy 会同步影响原对象（因为操作的是 `target`）。

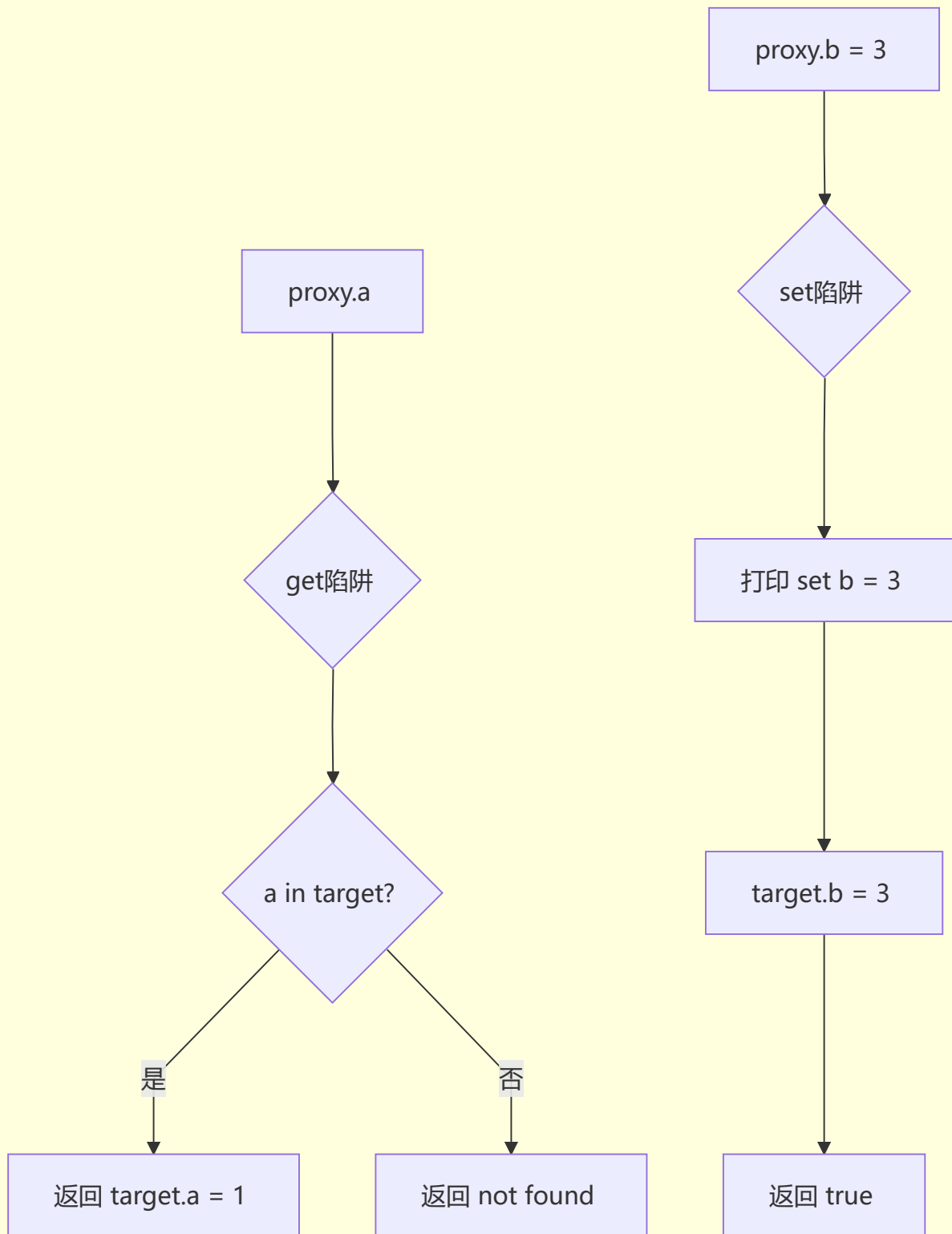
```
const obj = { a: 1, b: 2 };
const proxy = new Proxy(obj, {
  get(target, key) {
    return key in target ? target[key] : 'not found';
  },
  set(target, key, value) {
    console.log(`set ${key} = ${value}`);
    target[key] = value;
    return true;
  }
});
console.log(proxy.a);
console.log(proxy.c);
```

```
proxy.b = 3;  
console.log(obj.b);
```

输出:

```
1  
not found  
set b = 3  
3
```

Proxy 拦截流程



解析： Proxy 的 `get` 陷阱拦截属性读取，当 key 不在目标对象中时返回 'not found'。 `set` 陷阱拦截属性赋值，可以在赋值前后执行自定义逻辑。修改 proxy 会同步影响原对象（因为修改的是 target）。

2 Proxy vs defineProperty

💡 **要点:** Proxy 只能拦截通过 proxy 对象进行的操作。`proxy.arr` 触发 get 陷阱返回原数组引用后, `.push()` 直接在原数组上操作, 不再触发 Proxy 拦截。Proxy 相比 `Object.defineProperty` 能拦截更多底层操作 (如数组索引赋值)。

```
const data = { arr: [1, 2, 3] };
const proxy = new Proxy(data, {
  get(target, key) {
    console.log(`get: ${key}`);
    return target[key];
  }
});
proxy.arr.push(4); // 会输出什么?
```

输出:

```
get: arr
```

解析: Proxy 只拦截了 `get` 操作。`proxy.arr` 触发了 get 陷阱打印 "get: arr", 返回的是原数组引用。后续 `.push(4)` 直接在原数组上操作, 不会触发 Proxy 的 get 陷阱 (因为 push 内部通过索引访问数组元素是在数组内部进行的)。相比之下, `Object.defineProperty` 无法监听数组元素的增删 (push、pop 等), 而 Proxy 可以拦截 `[[Get]]`、`[[Set]]` 等底层操作。

3 Reflect 使用

💡 **要点:** Reflect 提供了一套操作对象的**规范化 API**。`Reflect.getPrototypeOf` 获取原型, `Reflect.ownKeys` 返回对象自身所有属性键 (包括不可枚举和 Symbol 属性)。

```
class Animal {
  constructor(name) {
    this.name = name;
  }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
}
// 使用Reflect检测
console.log(Reflect.getPrototypeOf(Dog) === Animal);
console.log(Reflect.ownKeys(new Dog('Buddy', 'Lab')));
```

输出:

```
true
['name', 'breed']
```

解析： `Reflect.getPrototypeOf(Dog)` 返回 Dog 的原型，即 Animal（因为 `class Dog extends Animal`）。
`Reflect.ownKeys` 返回对象自身的所有属性键（包括不可枚举和 Symbol 属性），Buddy 实例有 name 和 breed 两个自身属性。

九、Class 语法 & 继承

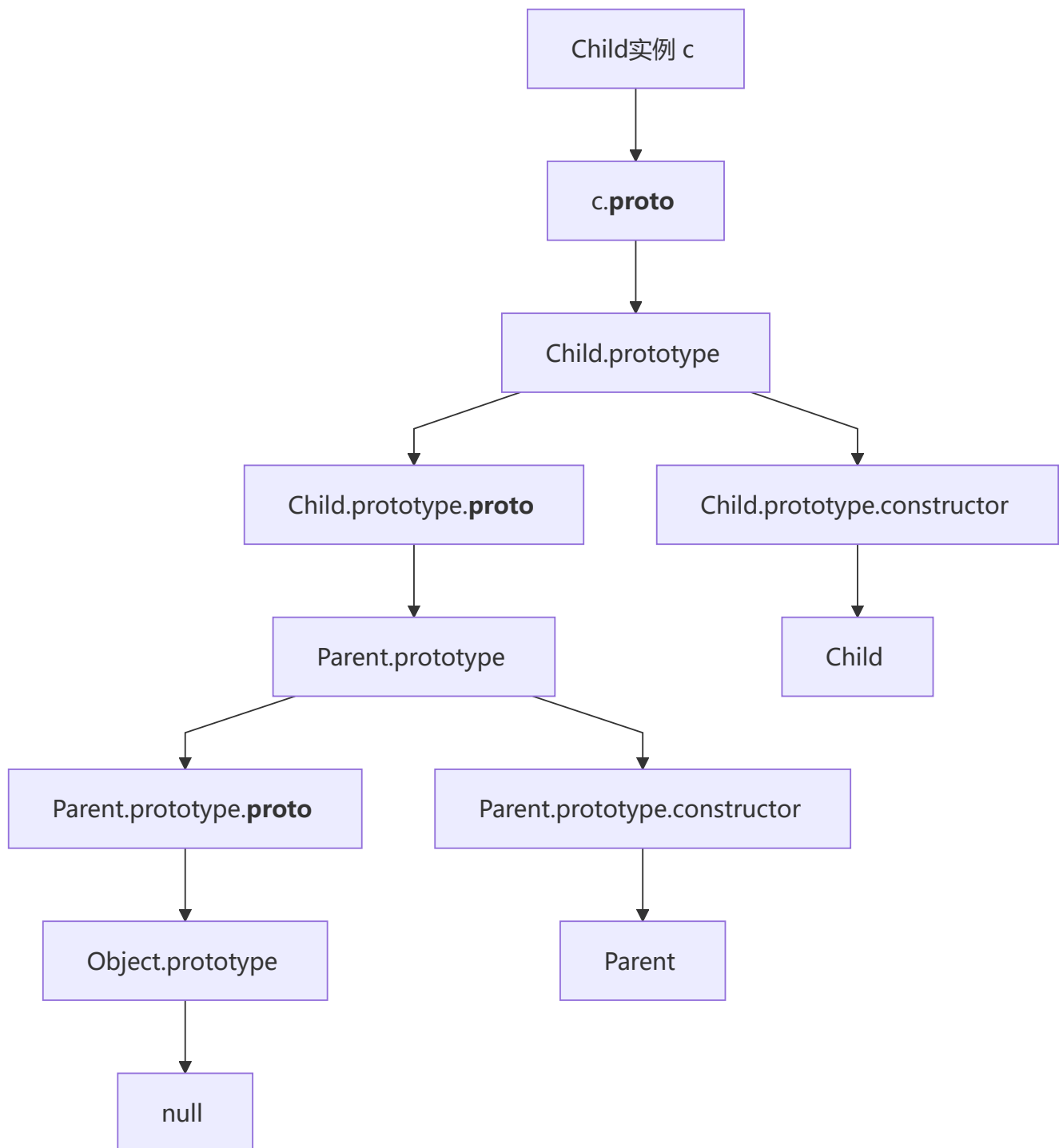
1 Class 基础

💡 **要点：** `extends` 实现继承，`super()` 必须在子类 constructor 中首先调用以初始化 this。子类会继承父类原型上的方法，`instanceof` 沿原型链检查。

```
class Parent {
  constructor() {
    this.name = 'parent';
  }
  sayHello() {
    console.log('Hello from ' + this.name);
  }
}
class Child extends Parent {
  constructor() {
    super();
    this.name = 'child';
  }
}
const c = new Child();
c.sayHello();
console.log(c instanceof Parent);
console.log(c instanceof Child);
```

输出：

```
Hello from child
true
true
```



解析： `super()` 必须在子类 constructor 中调用，它会调用父类的构造函数初始化 this。Child 类继承了 Parent 的 sayHello 方法，由于 this.name 在子类 constructor 中被覆盖为 'child'，所以输出 'Hello from child'。 `instanceof` 检查原型链，c 同时是 Child 和 Parent 的实例。

2 静态属性和私有字段

要点： `static` 属性定义在类本身（所有实例共享）。`#` 开头的私有字段是 JavaScript 的硬性私有机制，只能在类内部访问，外部访问会抛出语法错误。

```
class Foo {  
  static count = 0;  
}
```

```

#secret = 'hidden';

constructor() {
  Foo.count++;
}

getSecret() {
  return this.#secret;
}
}
const f1 = new Foo();
const f2 = new Foo();
console.log(Foo.count);
console.log(f1.getSecret());
console.log(f1.#secret); // 报错?

```

输出:

```

2
hidden
Uncaught SyntaxError: Private field '#secret' must be declared in an enclosing class

```

解析: `static count` 是类本身的属性，所有实例共享。创建了两个实例，count 为 2。`#secret` 是私有字段，只能在类内部通过 `this.#secret` 访问，`getSecret()` 方法可以正常返回。外部直接访问 `f1.#secret` 会抛出语法错误，这是 JavaScript 的**硬性私有**机制。

3 super 关键字

要点: `super()` 必须在子类 constructor 中使用 `this` 之前调用。执行顺序：先进入子类 constructor → 调用 `super()` 进入父类 constructor → 回到子类继续执行。

```

class A {
  constructor() {
    console.log('A');
  }
}
class B extends A {
  constructor() {
    console.log('B start');
    super();
    console.log('B end');
  }
}
new B();

```

输出:

```
B start
A
B end
```

解析：子类 constructor 中 `super()` 必须在 `this` 使用前调用（ES6 要求）。执行顺序是：先进入 B 的 constructor（打印'B start'），然后调用 `super()` 进入 A 的 constructor（打印'A'），最后回到 B 继续执行（打印'B end'）。这保证了父类的初始化先于子类的自定义逻辑。

十、Map / Set / WeakMap / WeakSet

1 Map 与 Object 区别

💡 要点：Map 的 key 可以是任意类型（包括对象、数字），而 Object 的 key 只能是字符串或 Symbol。Map 有 `size` 属性，可直接 `for...of` 遍历，且保持插入顺序。

```
const map = new Map();
const objKey = { id: 1 };
map.set(objKey, 'value');
map.set('key', 'value2');
map.set(42, 'value3');

console.log(map.size);
console.log(map.get(objKey));
console.log(map.has(42));

for (const [k, v] of map) {
  console.log(typeof k, v);
}
```

输出：

```
3
value
true
object value
string value2
number value3
```

解析：Map 的 key 可以是任意类型（对象、原始值），而 Object 的 key 只能是字符串或 Symbol。Map 有 `size` 属性直接获取元素数量，可以 `for...of` 直接遍历（Object 需要 `object.keys()` 或 `object.entries()`）。Map 保持插入顺序。

2 Set 唯一性

💡 **要点：**Set 中的值**唯一**（自动去重）。`NaN` 在 Set 中视为相等（尽管 `NaN !== NaN`），只添加一次。两个 `{}` 是**不同引用**，都会被添加。

```
const set = new Set([1, 2, 3, 3, 4, 4, 5]);
console.log([...set]);

set.add(NaN);
set.add(NaN);
set.add({});
set.add({});
console.log(set.size);
```

输出：

```
[1, 2, 3, 4, 5]
7
```

解析：Set 中的值具有唯一性，重复的 3 和 4 被自动去重。`NaN` 在 Set 中视为相等（虽然 `NaN !== NaN`），所以只添加一次。两个 `{}` 是不同的引用，所以都会被添加。最终 Set 包含：1, 2, 3, 4, 5, NaN, {}, {} → 共 7 个元素。

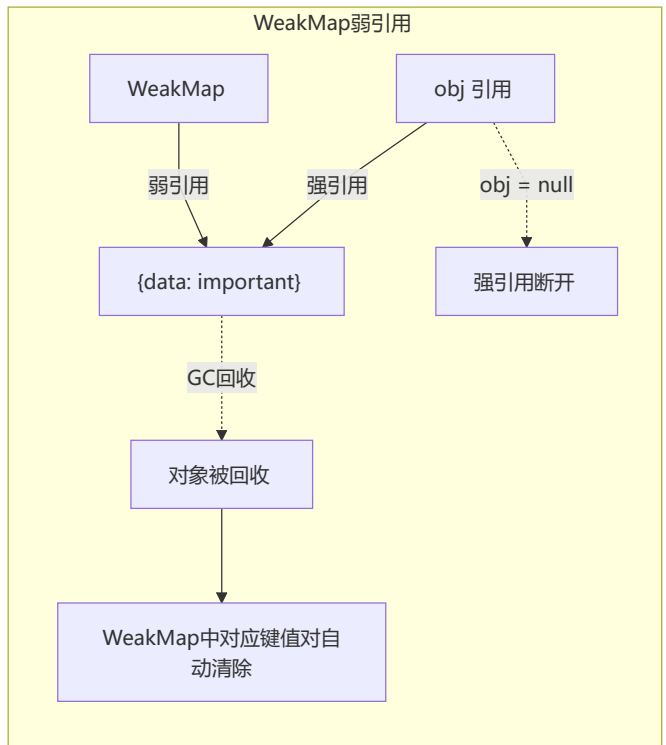
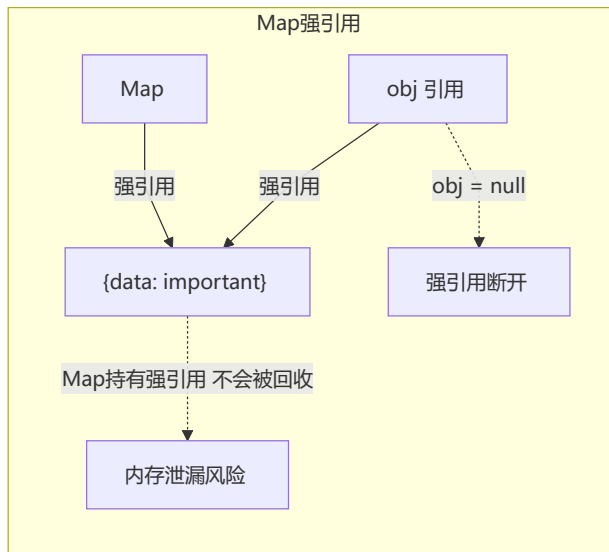
3 WeakMap 弱引用

💡 **要点：**WeakMap 的 key 是**弱引用**，不会阻止垃圾回收。当 `obj = null` 断开强引用后，对象被 GC 回收，WeakMap 中的对应键值对**自动清除**。WeakMap 不可迭代，无 `size` 属性，key 必须是对象。

```
let obj = { data: 'important' };
const wm = new WeakMap();
wm.set(obj, 'metadata');
obj = null;
// 此时WeakMap中的键会被GC回收吗？
console.log(wm.has(obj)); // ?
```

输出：

```
false
```



解析： WeakMap 的 key 是**弱引用**，不会阻止垃圾回收。当 `obj = null` 断开强引用后，对象没有其他强引用指向它，会被 GC 回收。此时 `wm.has(obj)` 传入 `null`，返回 `false`（且 WeakMap 已经自动清除了该键值对）。WeakMap 不可迭代，没有 `size` 属性，key 必须是对象。**注意：** `wm.has(obj)` 中 `obj` 已经是 `null`，WeakMap 的 key 只能是对象，所以这里实际传入了 `null`，返回 `false`。设计上 WeakMap 不允许原始值作为 key，传入非对象会抛出 `TypeError`，但 `null` 不会报错，而是返回 `false`。更准确地说，WeakMap 中已无该键值对，`wm.has(null)` 返回 `false`。

WeakMap 的主要用途： 存储 DOM 节点的元数据（节点被移除后自动清理，防止内存泄漏）、缓存私有数据。``